
Kubernetes Interview Q&A: Static Pods, Manual Scheduling, Labels & Selectors

Section 1: Static Pods

Q1. What is a static pod and how is it different from a regular pod?

Answer:

- A **static pod** is a pod managed **directly by the kubelet**, not the Kubernetes API server.
- **Key differences from regular pods:**

Feature	Static Pod	Regular Pod
Managed by	Kubelet	API Server + Scheduler
Scheduling	Node-specific, fixed	Scheduler decides node
Controllers	None	Can be managed by Deployment, ReplicaSet, etc.
Creation	YAML in <code>/etc/kubernetes/manifests/</code>	<code>kubectl apply</code>
Use Cases	Control plane components, debugging	Application workloads

Tip: Static pods are mirrored in the API server as **mirror pods** for visibility, but kubelet is the owner.

Q2. Where do you define static pods and how does kubelet manage them?

Answer:

- Static pod YAMLs are stored on a node, usually in:

`/etc/kubernetes/manifests/`

- Kubelet watches this directory:
 - **Add YAML** → pod created
 - **Delete YAML** → pod deleted
 - **Modify YAML** → pod restarted
 - **No scheduler or controllers are involved**; the pod always runs on that node.
-

Q3. Use cases of static pods in production

Answer:

1. Control plane components: kube-apiserver, kube-controller-manager, kube-scheduler
 2. Critical node-level services: logging, monitoring agents
 3. Cluster bootstrapping or debugging before the API server is functional
-

Q4. Limitations of static pods

Answer:

- No replication or self-healing across nodes
 - Node failure → pod is lost
 - Manual per-node configuration
 - Not suitable for scalable workloads
-

Q5. Practical scenario:

Question: You notice that a control-plane pod on a specific node is failing repeatedly. How would you troubleshoot static pods?

Answer:

1. SSH to the node.
2. Check `/etc/kubernetes/manifests/` for the pod YAML.
3. Inspect pod logs:

```
kubectl logs <pod-name> -n kube-system
```

4. Validate kubelet logs for errors:

```
journalctl -u kubelet -f
```

5. Make necessary changes to YAML and watch kubelet recreate the pod.
-

Section 2: Manual Scheduling

Q6. What is manual scheduling in Kubernetes?

Answer:

- Manual scheduling is **bypassing the Kubernetes scheduler** and assigning a pod to a specific node using the `nodeName` field in the pod spec.
- Example:

spec:

containers:

- name: nginx

image: nginx

nodeName: node1.example.com

- **No scheduler is involved**; the pod is created only on the specified node.
-

Q7. Use cases of manual scheduling

Answer:

1. Debugging a specific node
 2. Bootstrapping control plane pods
 3. Assigning critical services to known nodes
-

Q8. Risks/limitations

Answer:

- No fault tolerance (node failure → pod pending)
 - No replication
 - Pod tied to a single node (no portability)
 - Requires node awareness (`kubectl get nodes`)
-

Q9. Difference between manual scheduling, nodeSelector, and affinity

Feature	nodeName	nodeSelector	nodeAffinity
Scheduling	Manual only	Automatic	Automatic
Flexibility	✗ Fixed node	✓ Limited	✓✓ Full rules
Rescheduling	✗ No	✓ Yes	✓ Yes
Use case	Debug/static critical pod	Fixed environment pods	Complex placement policies

Q10. Practical scenario:

Question: You need to run a pod on a node that is under heavy load for testing. Which scheduling option would you use?

Answer:

- Use **manual scheduling** with **nodeName** to force the pod onto that node.
 - Optionally, use **taints/tolerations** or **node affinity** for more controlled testing.
-

Section 3: Labels & Selectors

Q11. What are labels and why are they important?

Answer:

- Labels are **key:value pairs** attached to objects like pods, nodes, services.
 - Used to **organize, group, and select resources**.
 - Enable **services, deployments, and scheduling** to target specific pods.
-

Q12. What are selectors in Kubernetes?

Answer:

- Selectors allow Kubernetes to **find objects based on labels**.
- Types:
 - **Equality-based:** match exact labels
 - **Set-based:** match multiple possible values

Example (equality):

selector:

matchLabels:

app: nginx

Q13. How are labels and selectors used in production?

Answer:

- **Services:** route traffic only to pods with matching labels
 - **Deployments/ReplicaSets:** manage pods via selector
 - **NetworkPolicies:** apply rules to a subset of pods
 - **Node scheduling:** nodeSelector or affinity
-

Q14. Practical scenario:

Question: You want a service to only route traffic to pods in staging environment. How would you configure it?

Answer:

spec:

selector:

app: myapp

env: staging

- Only pods with both labels (**app=myapp** and **env=staging**) will receive traffic.
-

Q15. Common pitfalls with labels & selectors

Answer:

1. Misaligned **selector** and pod labels → Deployment won't manage pods
2. Over-labeling → Hard to manage

3. Missing **version** label → Rolling updates become confusing
4. Changing labels of a running pod → Pod may be ignored by services/deployments

Q16. Can a static pod be scheduled on another node automatically if the original node fails?

Answer:

- No. Static pods are tied to the **node where the YAML exists**.
 - If the node goes down, the pod remains in a “Pending” state on the original node and will **not be rescheduled**.
 - For HA, you must manually place the YAML on another node.
-

Q17. Can a static pod be managed by a Deployment or ReplicaSet?

Answer:

- No. **Deployments, ReplicaSets, or StatefulSets cannot manage static pods**.
 - Static pods are **kubelet-managed only**.
 - In Kubernetes, static pods have **mirror pods** in the API server, but the API server does not control them.
-

Q18. How do you force a pod onto a specific node without using manual scheduling?

Answer:

- You can use **nodeSelector** or **node affinity** in the pod spec:

spec:

nodeSelector:

node-role.kubernetes.io/worker: ""

- This is **automatic scheduling with constraints**, unlike `nodeName`, which is hard manual scheduling.
-

Q19. Explain a scenario where manual scheduling is better than nodeSelector/affinity.

Answer:

- **Debugging a failing node:** You want to run a specific pod only on that node to test resources or troubleshoot kubelet behavior.
 - **Bootstrap a control plane pod:** Before scheduler is functional, you can manually place API server or controller manager pods on a node.
-

Q20. What are mirror pods in Kubernetes?

Answer:

- Mirror pods are **API server representations of static pods**.
 - Kubelet creates a mirror pod in the API server so that tools like `kubectl get pods` can see static pods.
 - **OwnerReference** points back to kubelet, not a controller.
-

Q21. Can labels affect scheduling of pods?

Answer:

- Yes. Labels combined with **nodeSelector**, **node affinity**, and **taints/tolerations** determine where a pod can run.
- Example:

spec:

nodeSelector:

env: production

- Only nodes with **env=production** label will be eligible for scheduling.
-

Q22. How would you perform a rolling update using labels?

Answer:

1. Create a Deployment with pods labeled by version: **version=v1**.
2. Update the Deployment YAML to **version=v2** in the pod template.
3. Kubernetes will:
 - Create new pods with **version=v2**
 - Gradually terminate old pods with **version=v1**
4. Services use labels to route traffic automatically.

Labels allow **canary, blue/green, or incremental rollouts**.

Q23. How do you identify static pods in a cluster using kubectl?

Answer:

- Use:

```
kubectl get pods -A -o wide
```

- Static pods usually:
 - Are in **kube-system** namespace

- Pod name is `<pod-name>-<node-name>`
 - OwnerReference points to **kubelet**, not a Deployment/ReplicaSet
-

Q24. Can you combine static pods with manual scheduling?

Answer:

- Yes. Static pods are inherently node-specific, so the **node where the YAML exists** acts as the “manual scheduling.”
 - You can also add **labels** to static pods for services, network policies, or monitoring.
-

Q25. Describe a failure scenario involving labels and selectors. How would you fix it?

Scenario: A service routes traffic to pods labeled `app=nginx`, but traffic is not reaching new pods.

Answer:

1. Check pod labels:

```
kubectl get pods --show-labels
```

2. Ensure pods have `app=nginx`.
3. Verify service selector:

```
kubectl describe svc <service-name>
```

4. Update Deployment/pods with missing label:

```
kubectl label pod <pod-name> app=nginx --overwrite
```

Lesson: Misaligned labels/selectors are a common root cause for traffic routing or deployment issues.

✅ Bonus Advanced Questions

1. **Q:** Can static pods use labels and be selected by services?
A: Yes, static pods appear as **mirror pods** in the API server and can be targeted by services via labels.
 2. **Q:** Can you combine manual scheduling with labels?
A: Yes, you can assign **nodeName** (manual scheduling) and labels to allow services or network policies to target that pod.
 3. **Q:** How do you rollback a Deployment when pods are labeled with different versions?
A: Kubernetes uses **ReplicaSet history** to revert to previous pods with the desired labels. Labels help track versions for safe rollbacks.
-